PHANTOM: Causal Attribution of Runtime SBOM Drift via eBPF Behavioral Contracts in Kubernetes

Harsh Ranjan
Vellore Institute of Technology
Email: harsh.ranjan2023@vitstudent.ac.in

Jai Goel
Vellore Institute of Technology
Email: jai.goel2023@vitstudent.ac.in

Dr. Thanuja R
Guide, Vellore Institute of Technology
Email: thanuja.r@vit.ac.in

Abstract—Cloud-native applications rely on third-party components whose runtime behavior can change after deployment. Monitors and software bills of materials (SBOMs) capture different evidence; linking events to components and causal responsibility remains difficult. PHANTOM implements SBOM verification interfaces, eBPF event collection, variable-order Markov behavioral contracts, a time-decayed Behavioral Dependency Graph (BDG), and a DoWhy-based causal-estimation pathway and an XGBoost PCEPS scorer. Its design combines image- and PURL-scoped contract generation with a temporal causal projection that returns not_identifiable when specified identification checks fail. This is an integration design, not a claim that these mechanisms are new or that component-level causal attribution has been validated. In an evaluation of six completed attack runs across the two tested supply-chain attack families, PHANTOM recorded drift events with mean latency 15.74 seconds, median 5.52 seconds, and linearly interpolated 95th percentile 54.72 seconds. Three SolarWinds-style injections failed before an attack phase. The saved event records lack resolved PURL/SBOM bindings, and the dataset contains no benign controls; thus attribution accuracy, false-positive rate, calibrated prediction, and runtime overhead remain unestablished. These preliminary results document event presence and latency only. PHANTOM contributes an implementable, uncertainty-aware research pathway whose component mapping and causal value require controlled evaluation.

Index Terms—eBPF, software supply chain, causal inference, Kubernetes, runtime security, SBOM

I. Introduction

Cloud-native applications assemble services from third-party packages, container images, build tools, and deployment configuration. That composition shortens development cycles, but it also creates several points where the delivered software can differ from what an operator expected. A dependency may be substituted, a build artifact may be altered, or a legitimate update may change runtime behavior. An SBOM records component inventory and relationships; runtime monitoring observes what executes. Neither view alone answers whether an observed process, file access, or network connection came from a particular package or whether a package change caused the behavior.

The XZ Utils incident illustrates the problem. Modified build scripts introduced malicious functionality into selected liblzma release artifacts, and the resulting behavior affected an OpenSSH authentication path. The compromise became public after Andres Freund investigated anomalous CPU use and related symptoms in SSH testing [1], [2]. A runtime event such as an unexpected execution or network connection could be useful evidence in a similar incident, but that event alone would not identify the responsible package. A process may load code from several packages, inherit configuration from its workload, or perform unusual behavior after a legitimate deployment change. Attribution therefore depends on provenance mapping and explicit assumptions about the causes of observed behavior.

For example, when a runtime monitor reports an unexpected execution, its event record may identify the process and operation. The record does not by itself establish whether code associated with ‘pkg:pypi/lz4@4.3.2‘ caused the execution or whether a configuration change in the same container explains it. PHANTOM investigates an architecture for joining these evidence types: contracts keyed by image digest and PURL, event relationships in a Behavioral Dependency Graph (BDG), and a causal-estimation pathway that abstains when configured identification checks fail. The available attack records do not contain resolved PURL/SBOM bindings, so the intended component-level link is implemented as a system pathway but is not demonstrated by these experiments.

This paper makes three implementation and evaluation contributions, with evidence scopes stated separately:

- Implemented and observed at the event level: the repository contains a variable-order Markov contract-generation pipeline, BIC order selection, and KL-divergence scoring. In six completed attack runs across two families, the saved records contain drift events within the recorded detection windows; their median injection-to-event latency is 5.52 s. The records do not establish that those events were bound to the responsible SBOM component.
- Implemented and tested structurally: the repository contains a typed, time-decayed BDG and a temporal DAG projection with a fixed semantic tier ordering. The local test run completed with 454 passed and one skipped test across the core library and five services. These unit tests exercise code paths, including cycle handling and abstention cases; they do not prove causal correctness or component attribution in de-

ployment.

- Implemented, awaiting experimental validation: a DoWhy adapter and a 16-feature PCEPS training, calibration, and scoring pathway are present. The adapter returns not_identifiable for specified failures rather than using a correlation fallback. The saved run artifacts contain no component-level oracle labels, trained scoring model, or held-out predictive results.

A. Design Distinctions and Research Claims

PHANTOM's research question concerns the interfaces among event collection, package identity, behavioral deviation, and causal analysis; the use of eBPF, Markov models, graphs, DoWhy, or XGBoost alone is not claimed as novel. The implementation makes three testable design choices. First, contract-generation requests carry an image digest and PURL, enabling component-scoped model records when event-to-component identity is available. Second, the causal projection orients selected within-window edges using an explicit semantic tier hierarchy, while preserving the underlying BDG and withholding queries that remain cyclic. Third, identification failures produce the first-class status not_identifiable, rather than an association score presented as a causal effect. These are implementation properties and candidate research contributions, not a verified claim that PHANTOM is the first system to make them or that the tier assumptions recover true causal structure. The saved experiment records lack the package bindings and independent labels needed to test whether these choices improve attribution.

The evaluation is intentionally narrower than a detector benchmark. Six completed attacks produced event-positive records; three SolarWinds-style injections failed before an attack phase. No benign control windows were saved, so false-positive rate and precision cannot be calculated. The baseline log documents a dry run rather than comparable Falco, Trivy, or Isolation Forest results. Mean latency was 15.74 s, median 5.52 s, and linearly interpolated P95 54.72 s; the first XZ run took 70.332 s, and the available logs do not explain the delay. Reporting these limits makes clear which observations the present artifacts support and which research questions remain open; it does not turn the six-run dataset into evidence of general detection performance.

Section II reviews supply-chain attacks, runtime telemetry, SBOMs, and causal analysis. Section III describes the threat model, contract pipeline, BDG, causal projection, and PCEPS. Section IV summarizes the services and analyst interface. Section V reports the saved evaluation evidence and its limitations, including the controls and baselines still needed. Section VI concludes with the implementation contribution and the next validation steps.

II. Background and Related Work

A. Software Supply Chain Attacks

Supply-chain compromises exploit trust relationships between producers, build systems, and consumers. SolarWinds disclosed SUNBURST in December 2020 after malicious code had been inserted into the Orion build pipeline and distributed inside digitally signed updates. The backdoor remained dormant before initiating selective network communications, and its delivery through a trusted vendor update complicated detection based on package reputation alone [3], [4]. Log4Shell, disclosed in December 2021 as CVE-2021-44228, was different: attacker-controlled strings could trigger JNDI lookups in vulnerable Log4j configurations, potentially leading to remote code execution. The library was widely embedded transitively, so asset owners often did not know which services loaded it [5], [6].

The XZ Utils compromise became public on March 29, 2024, when a developer found anomalous CPU behavior during SSH testing. Modified build scripts enabled a backdoor in selected release artifacts, despite source review and package provenance appearing ordinary to many downstream consumers [1], [2]. These incidents differed in mechanism, but each challenged a different monitoring assumption: SUNBURST used delayed, selective execution; Log4Shell was activated by input and dependency reachability; XZ altered build outputs and targeted a sensitive library interaction. At the kernel boundary, each could produce observable evidence such as unexpected process execution, file access, or outbound connections, but no single event signature would uniquely establish the responsible package. Kernel evidence can narrow the explanation; it cannot by itself prove software provenance.

B. eBPF-Based Runtime Security

Falco uses eBPF in cloud-native systems to monitor Linux and report suspicious activity [7]. It lets users create rules through a policy rule engine, which is useful for recognizing known malicious patterns and policy violations. Its coverage, however, depends on the rules available: behavior outside the configured rules may go undetected.

Tetragon also uses eBPF to monitor operating-system activity, including process execution and network use, for security observability and policy enforcement [8]. It provides Kubernetes-aware controls, while linking its events to a specific SBOM component requires additional identity mapping and analysis.

Tracee provides Linux runtime security and forensic capabilities using eBPF, including event collection and policy-based detection [9]. These tools can provide useful evidence and enforcement, while package-level attribution depends on the deployed configuration and any identity-mapping integrations. Collecting events or enforcing a policy does not by itself estimate whether a particular component change caused an observed deviation.

C. Software Bill of Materials and Runtime Drift

CycloneDX defines a machine-readable bill of materials for software components, dependencies, and related supply-chain metadata [10]. The NTIA minimum-elements work also established a practical baseline for SBOM fields and exchange [11]. Such formats make component inventories more useful, but a manifest remains a statement about declared or observed contents at a point in the build or scan process. It is not a continuous record of behavior after deployment.

Prior work has connected runtime evidence and SBOM enforcement. Bomfather uses kernel-level eBPF observation during builds to identify dependencies and construct build artifacts; its scope is build-time dependency discovery rather than the post-deployment drift and causal-estimation workflow studied here [29]. Crawford emphet al. describe enforcing SBOM constraints through Linux eBPF and IMA [30]. These efforts make clear that runtime/SBOM integration is not, by itself, PHANTOM's novelty. The repository instead implements a combination of image/PURL-scoped behavioral contracts, a typed time-decayed graph, and an explicit causal-estimation eligibility pathway; the current evaluation does not establish that this combination improves attribution.

Anchore Enterprise uses SBOM-centered analysis for vulnerability and supply-chain management [12], while Ox Security describes software supply-chain risk analysis across development and production contexts [13]. Those capabilities address important inventory and risk questions. PHANTOM focuses on a narrower gap: the package can remain in the SBOM under the same name and version while its runtime behavior changes. A reliable response therefore needs both artifact identity and live behavior, with uncertainty retained when an event cannot be mapped confidently to a component.

This distinction matters because package identity is not a behavioral invariant. A PURL normally identifies package type, namespace, name, and version; it does not prove that a deployed file is byte-for-byte identical to reviewed source or artifact. Digest pinning and signature verification can establish stronger provenance, but even a validly signed package may contain harmful code, as supply-chain incidents have demonstrated. Runtime observation complements those controls by checking what loaded software does in the environment where it executes. It cannot replace provenance: an event-to-PURL association is only as reliable as the image metadata, process mapping, and file attribution used to make it. A practical system must preserve these provenance links and expose missing links to the analyst rather than silently assigning an event to the nearest package.

The same distinction shapes PHANTOM's handling of benign change. Library upgrades, feature flags, configuration changes, and workload autoscaling can all alter event frequencies without indicating compromise. A contract trained on one operating envelope may therefore generate alerts when a service changes legitimately. Versioned contracts, workload context, and analyst review are required to separate routine evolution from suspect drift. We treat contract violation as evidence requiring explanation, not a verdict that a package is malicious. This policy is particularly important for services whose legitimate behavior is adaptive or customer-specific.

D. Causal Inference Applied to Security

Cloud incident root-cause systems combine operational signals to help explain service incidents; RCACopilot, for example, uses an LLM to predict a root-cause category and produce an explanation from incident diagnostics [14]. Earlier host-telemetry research programs also study system-level evidence and provenance [18]. Graph-based anomaly detection similarly uses relationships among entities to identify patterns that are difficult to see in isolated events. An anomaly score or ranked explanation does not by itself identify an intervention effect.

PHANTOM uses Pearl's intervention notation and DoWhy's causal workflow to represent assumptions behind a component-level estimate [15], [16]. The distinction is between observing a component substitution and intervening on that substitution while holding an identified adjustment set fixed. Applying a general causal-inference library to runtime evidence is an integration; novelty depends on the graph construction, identification checks, mapping quality, and evaluation rather than the use of DoWhy alone.

SBOM generators establish inventories; vulnerability scanners connect inventories to advisories; runtime monitors collect events and apply policies; incident tools correlate operational signals. PHANTOM targets the interfaces among these layers. It does not replace provenance verification, vulnerability management, or runtime enforcement. The repository uses PURL identifiers as component keys, following the package URL specification [26].

Contract generalization remains an open question. Evaluation should compare contracts keyed only by PURL with models conditioned on workload role or service mode, and should compare temporal Markov models with frequency baselines using matched event access.

E. Comparison with Existing Work

Table I summarizes documented capability categories for representative systems. Product features depend on deployment and integrations. PHANTOM's entries describe the repository implementation rather than empirically validated performance.

This table compares stated capabilities, not measured effectiveness. A performance comparison would require concurrent runs on identical workloads with the same attack and benign-control labels. The available baseline log records a dry run: Falco and Trivy setup were skipped,

TABLE I
Capability comparison against related systems (design-level comparison, not a performance benchmark)

System	eBPF collection	SBOM awareness	Runtime behavior model	Causal attribution	Pre-compromise prediction
Falco	Yes	No	Rules	No	No
Tetragon	Yes	No	Policies/events	No	No
Tracee	Yes	No	Policies/events	No	No
CycloneDX / Anchore	No	Yes	No	No	Vulnerability/risk analysis
Bomfather	Yes (build-time)	Yes	Build dependency observation	No	No [29]
eBPF + IMA	Yes	Yes	SBOM enforcement	No	No [30]
KubeArmor	eBPF/BPF	No	Policies	No	No [19]
LLM-based RCA	Varies	No	Incident signals	Ranked explanation	Varies
PHANTOM	Yes	Yes	Component contracts	DoWhy pathway	PCEPS implementation

Capabilities reflect documented design goals; deployment-specific integrations may extend individual tools beyond what is shown.

Falco alert output is absent, and the Isolation Forest result used synthetic frequency data rather than the saved attack traces. Trivy performs static vulnerability analysis and would require a separately defined, task-appropriate outcome. The current repository artifacts therefore support no claim that PHANTOM outperforms these tools. A representative workload such as Online Boutique [17] could be used in a future controlled comparison, but it is not part of the six-run dataset.

F. Gap This Paper Addresses

The existing system categories address different parts of the problem: runtime event collection and policy enforcement, component inventory and provenance, or incident-level diagnosis. PHANTOM explores an interface among these categories by representing image/PURL-scoped contracts, runtime relationships, and an explicit causal-query abstention status in one implementation. This paper does not claim that no published system offers any of these capabilities, nor that the combination has been proven to improve attribution. The empirical question is whether resolved component identity and a defensible temporal causal model can support more accurate, useful explanations than event presence or graph ranking alone. The current records cannot answer that question because they lack PURL-bound events and component-level ground truth.

III. System Design

A. Threat Model

PHANTOM detects runtime drift associated with a software component whose behavior has changed relative to a learned or approved contract. This includes a malicious replacement that retains a familiar package name and version, dependency confusion that introduces an unexpected package into a workload, build-pipeline tampering that produces behavioral changes, and backdoored libraries that preserve their public API. The deployment maps workload artifacts and SBOM PURLs to observed processes, enabling event-level evidence to be associated with component identities.

B. Behavioral Contract Generation

For a component identified by a PURL, the contract-generation use case retrieves training sequences scoped by image digest, PURL, and time interval. A token is a four-tuple $\tau(e) = (t, o, r, p)$ for event type, operation class, resource class, and privilege class. The Markov model operates over the observed token alphabet plus reserved unknown/resource tokens. The implementation does not by itself establish that the training interval contains only normal behavior; that interval and its approval must be controlled by the operator.

For each context h, the model estimates next-token probabilities. Candidate orders are compared using BIC with $q_k = |H_k|(|\Sigma| - 1)$ parameters, and contexts are pruned when their local BIC does not improve over a retained suffix. Laplace smoothing is

$$\hat{p}(x \mid h) = \frac{n(h, x) + 1}{n(h) + |\Sigma|}, \qquad \epsilon_h = \frac{1}{n(h) + |\Sigma|}. \tag{1}$$

For each observed context h, the scorer computes conditional KL divergence and weights it by the fraction of events observed in that context:

$$D_{\mathrm{KL}} = \sum_h \frac{n_h}{N} \sum_x \widehat{P}_{\mathrm{obs}}(x \mid h) \log \frac{\widehat{P}_{\mathrm{obs}}(x \mid h)}{P_{\mathrm{contract}}(x \mid h)}. \tag{2}$$

The implementation reports novelty separately and applies an optional event-loss multiplier. The scorer does not implement the split-conformal threshold described in the handoff, so this paper does not claim a calibrated binary drift decision or false-alarm guarantee. Contract serialization is hashed, signed through the cosign signer port, and persisted.

The contract model records image digest, PURL, model digest, signature bundle, signing identity, selected order,

Algorithm 1 Behavioral contract construction

Require: Sequences partitioned by $(d, p, [t_0, t_1])$, image digest d, PURL p, token sequences S, minimum $N_{\min}$, maximum order $K_{\max}$
Ensure: Signed model $C_{d,p}$ or failure
$N \leftarrow$ total number of tokens in S
if $N < N_{\min}$ then
    return failure
end if
$\Sigma \leftarrow$ observed tokens plus reserved unknown/resource tokens
for $k = 0$ to $K_{\max}$ do
    Estimate order-k transition counts and BIC score
end for
$k^* \leftarrow \arg\min_k \mathrm{BIC}(k)$
for all retained histories h do
    $\hat{p}(x \mid h) \leftarrow (n(h, x) + 1)/(n(h) + |\Sigma|)$
end for
Bind model to d, p and provenance metadata; serialize and hash
Sign serialized model and persist contract record
return $C_{d,p}$

BIC, training count, and creation time. Training data or signature integrity does not establish baseline correctness. In evaluation, event identities that cannot be resolved must remain unattached to a PURL.

1) Complexity: Let N be the total training-token count, $K_{\max}$ the maximum candidate order, Q the number of distinct context-token count entries across candidate orders, H the number of retained contexts, and $|\Sigma|$ the alphabet size. Transition counting costs $O(NK_{\max})$ in the worst case. Context sorting, pruning, and parent lookup add up to $O(Q \log Q + QK_{\max})$; materializing smoothed probabilities costs $O(H|\Sigma|)$. Thus a conservative bound is $O(NK_{\max} + Q \log Q + QK_{\max} + H|\Sigma|)$ time and $O(Q + H|\Sigma|)$ space. At inference, forming each context costs up to $O(K^*)$ for selected order K^*, followed by expected constant-time count lookups and evaluation of observed context-token pairs. A window of length ℓ therefore costs $O(\ell K^* + Q_w)$ time and $O(Q_w)$ temporary space, where Q_w is the number of distinct observed context-token pairs. No sub-millisecond runtime claim is made because the available artifacts contain no isolated scoring benchmark.

C. Behavioral Dependency Graph

We define the BDG as $G = (V, E, \phi, \psi)$, where V is a typed entity set, E is a typed multiedge set, ϕ maps nodes to observed attributes, and ψ maps edges to event time, confidence, and weight. Node types include workload, container, process, PURL, file, network endpoint, contract, and drift event. Implemented relations include runs, executes, belongs_to, reads, writes, connects_to, derived_from, and violates. Multiple edges are retained; the implementation uses a NetworkX MultiDiGraph representation.

At each observation time, an edge weight decays according to

$$w_e(t) = \lambda^{(t - t_{\mathrm{prev}})/\Delta}\, w_e(t_{\mathrm{prev}}) + q_e, \tag{3}$$

where $0 < \lambda < 1$ controls forgetting, Δ is the time scale, and q_e is the new evidence contribution. The temporal projection assigns semantic variables to six implemented tiers: environment/RBAC/image (0), component version (1), process behavior (2), file/network/privilege behavior (3), contract deviation (4), and runtime SBOM drift (5). In a single time window, the implementation permits an edge only from a lower to a higher tier; cross-window edges point from the earlier to later window. This fixed ordering is a modeling assumption motivated by the hypothesized supply-chain sequence, not evidence that the ordered variables are causally related in every workload. It can orient simultaneous observations to yield a DAG, but does not by itself establish identification or causal truth. The BDG retains cycles because runtime interactions can be cyclic. Projection code conservatively excludes unsupported orientations and withholds attribution when an unresolved cycle involves the treatment or outcome.

D. Structural Causal Attribution

The estimator represents a candidate component change as treatment S, a binary drift outcome as D, and measured pre-treatment covariates as X. The target is an average treatment effect $\mathbb{E}[D \mid do(S = 1)] - \mathbb{E}[D \mid do(S = 0)]$ under the supplied causal graph and adjustment set. The outcome follows from the input causal observations; covariates are supplied by the caller.

The BDG projection expands semantic variable classes across a fixed number of windows. Edges are added across strictly increasing time windows, or within a window when a hard-coded semantic tier increases. The SCM builder then collapses window labels to semantic variable names, adds treatment-to-outcome and covariate-to-treatment/outcome edges, checks variation and a positivity rule, and serializes the graph for DoWhy. DoWhy identification is requested with unidentifiable effects disallowed [15], [16]. The implementation returns not_identifiable on treatment/outcome cycles, no variation, positivity failure, construction, identification, or estimation errors. Tests exercise cycle and no-variation abstention paths; these tests validate software behavior under constructed inputs, not the correctness of causal assumptions in deployment.

For completed models, the configured estimator returns an ATE and, when available, an interval. The pathway attempts a treated-cohort counterfactual and runs random-common-cause, placebo-treatment, and data-subset refutations. Its confidence value is a heuristic product involving event-loss rate, identity resolution, contract verification, interval width, and refutation pass fraction; it is not a

TABLE II
Implemented PCEPS feature schema and source

Slot	Feature	Input source
1	ATE-derived causal effect	Causal attribution result
2	Attribution confidence	Causal attribution result
3	Contract-violation rate	Contract validator
4	Behavioral divergence ratio $D/\max(\theta,\delta)$	KL drift scorer and contract threshold
5	New-process rate	Process events and contract
6	Unexpected-network rate	Network events and contract
7	Unexpected privilege transition	Privilege events and policy context
8	Sensitive-file violation rate	File events and policy context
9	Component criticality	Component policy input
10	Invalid image signature	SBOM/image verification status
11	Namespace risk weight	Namespace policy input
12	Service-account privilege	Kubernetes RBAC context
13	Event-loss rate	Collector loss counters
14	Graph-centrality delta	BDG snapshot and baseline
15	Prior drift frequency	Prior scored windows
16	Runtime-component novelty	Runtime-to-SBOM identity resolution

calibrated probability of causal correctness. The graph, covariates, and unmeasured confounding assumptions govern interpretation. The run artifacts contain no independent component-level ground truth, so attribution accuracy is not reported.

E. PCEPS Risk Scorer

PCEPS is a separate analyst-triage pathway, not an attack detector. Table II lists the implemented feature schema. Features are clamped to $[0,1]$; missing features are represented with a mask and imputed from training medians, except selected conservative defaults that use one. The scorer concatenates 16 values and 16 mask indicators, obtains an XGBoost output p [24], and applies the implemented Platt mapping [25]:

$$p_{\mathrm{cal}} = \sigma(a\,\mathrm{logit}(\mathrm{clamp}(p)) + b)\,, \qquad \mathrm{PCEPS} = 100 p_{\mathrm{cal}}. \tag{4}$$

The implementation clamps p to $[10^{-7}, 1-10^{-7}]$ before the logit; the resulting score lies in $[0,100]$ and is mapped to fixed severity bands at 20, 40, 60, and 80. Although p_{cal} is a classifier probability output, the available project artifacts do not establish calibration or future-compromise prediction. PCEPS is intended to prioritize review when multiple indicators co-occur; a high score does not establish compromise, causal responsibility, or calibrated probability. Predictive validity remains untested because no trained model, held-out labels, or calibration report accompanies the run artifacts.

IV. Implementation

The repository contains SBOM, eBPF, causal-engine, API-gateway, and report-generator services, with a React/TypeScript analyst interface. The SBOM service integrates Syft and Cosign interfaces for generation and verification [20], [21], [27]. The README documents Python 3.11+, Docker Compose, Terraform/Helm, and Linux CO-RE requirements [22], [28]; the infrastructure tree includes an AWS EKS deployment. A repository URL and dataset DOI were not available in the checked-in materials.

The DoWhy adapter delegates to the causal use case and maps identification and estimation failures to not_identifiable; no correlation fallback is used as a causal estimate.

The API gateway exposes operations to the analyst client, and the report generator assembles evidence for later review. Event schemas are versioned across service boundaries.

The graph implementation uses NetworkX's directed multigraph representation [23]. The D3 graph view exposes graph nodes, edges, and evidence records for analyst inspection. Its visualization does not add independent evidence for a causal claim.

V. Evaluation

A. Experimental Setup

The checked-in raw3 analysis set contains nine JSON records dated September 5, 2026. Six records contain a completed attack phase: three ‘xzutils-001‘ runs and three ‘dep-confusion-001‘ runs. Three SolarWinds-style ‘solarwinds-001‘ records report ‘inject() returned False‘ and have no attack phase. They are failed trials, not detection outcomes. The successful records include injection and first-event timestamps, raw drift-event records, and phase metadata. The recorded detection window is 120 s after injection, as specified in the raw scenario records and evaluation code. The older ‘raw/‘ directory contains 114 legacy JSON records, including entries labelled for benign scenarios, but the records inspected have no PHANTOM detections and include ‘dry-run-pod‘ placeholders; they cannot serve as live benign controls. Although the run script describes a planned EKS setup, the raw3 files do not provide a complete cluster/kernel/container/image-digest manifest. The saved event records identify a cluster name but have ambiguous or missing pod identity and no SBOM binding.

B. RQ1: Detection Performance

All six completed attack runs are marked event-positive because at least one drift event was recorded within the detection window. This gives an event-positive fraction of 6/6 (1.000; Wilson 95% interval [0.610, 1.000]). It does not establish component-level attack detection: the recorded events are namespace-change or privilege-transition events, and their runtime identity is ambiguous or missing, with no PURL or SBOM ID. The three failed SolarWinds-style injections do not count as false negatives. The evidence therefore supports only event-presence counts for the two completed families.

Latency is measured from injection timestamp to first recorded PHANTOM event. The recomputed mean is 15.737 s, median 5.516 s, and linearly interpolated P95

TABLE III
Experimental configuration and run manifest

Parameter	Evidence in saved artifacts
EC2 instance type	Not recorded
Linux kernel	Not recorded in raw3 manifest
Kubernetes version	Not recorded in raw3 manifest
Container runtime/version	Not recorded
Experiment commit	Not linked to raw3 records
Python / DoWhy versions	Repository requires Python 3.11+; experiment versions are not recorded
Completed attacks	xzutils-001: 3; dep-confusion-001: 3
Failed injections	solarwinds-001: 3 (‘inject() returned False‘)
Detection window	120 s post-injection
Baseline outputs	Dry-run record only; no comparable raw detections
Benign controls	None in current dataset
PURL/SBOM binding	Unresolved in saved event records
CPU/memory overhead	Not measured; saved snapshots are zero without provenance

TABLE IV
Run-level outcomes by attack family

Attack family	Valid	Event+	Failed
XZ-style backdoor	3	3	0
Dependency confusion	3	3	0
Build-pipeline tampering	0	0	3
Benign controls	0	N/A	N/A

Note: Completed attack runs had unresolved runtime identity; all three build-pipeline injections failed; no benign controls were recorded.

54.718 s, rounding to 15.74, 5.52, and 54.72 s. Each stored MTTD matches its timestamps. The first XZ run has a 70.332 s latency; available records do not establish its cause. The event rows are duplicated in the saved API output, so the unit is the scenario run, not 3,701 event rows.

The listing connects implemented service pathways; it does not imply an end-to-end evaluation. In particular, the drift scorer has no implemented alert threshold, causal eligibility is conditional on the supplied data and graph assumptions, and the saved experiment set contains no trained PCEPS artifacts.

Because the dataset contains no benign controls, FPR, specificity, precision, F1, and false positives per hour are undefined. These values are recorded as unavailable rather than set to zero. The six-run Wilson interval is wide; broader evaluation is needed before generalizing beyond the tested runs.

C. RQ2: Causal Attribution Accuracy

The checked-in aggregate reports attribution accuracy of 1.0, but the underlying records do not contain per-run predicted PURLs or a comparison with the oracle target PURL. All saved drift events have ambiguous or missing identity, null PURL/SBOM fields, and unknown workload identifiers. Component identity and causal correctness therefore cannot be verified, and no attribution accuracy is reported.

Algorithm 2 Evidence-gated PHANTOM processing pathway

Require: SBOM and image identity evidence; normalized event window W; available contract, BDG, causal, and PCEPS artifacts
Ensure: Evidence report with explicit unavailable or unresolved fields

```
Verify SBOM digest and signature; reject invalid artifacts
Normalize supported events and resolve workload/process identity
if component identity is supported by verified evidence then
    associate events with image digest and PURL
else
    retain unresolved identity; do not assign a PURL
end if
if a valid contract is available then
    compute the implemented KL score and novelty indicators for W
else
    mark contract comparison unavailable
end if
update the typed BDG, ignoring duplicate event UUIDs and applying edge decay
if causal observations satisfy implemented graph and data eligibility checks then
    estimate the configured DoWhy effect and run available refutations
else
    return not_identifiable with the diagnostic reason
end if
if trained PCEPS model and calibration parameters are available then
    calculate the bounded score from the 16-feature vector and missingness mask
else
    mark PCEPS unavailable
end if
generate a report from recorded evidence and unresolved fields
```

D. RQ3: PCEPS Prediction

The PCEPS source implements feature extraction, XGBoost training, Platt scaling, Brier/ECE calculations, and scoring. No trained PCEPS model, calibration artifact, scored windows, labels, or metric output supporting the manuscript's former AUC/Brier values was found. No predictive-performance result is reported. The score's future-outcome interpretation requires a separate prospective evaluation.

E. RQ4: Overhead

The saved CPU, memory, and event-lag snapshots are all zero. No raw Prometheus samples or measurement

provenance accompany them. These records cannot establish zero resource cost or a particular event-loss rate; runtime overhead remains unmeasured.

F. Proposed Benign-Control Protocol

Benign controls are required to estimate how often ordinary workload changes trigger the event or contract pipeline. A future study should include routine package updates, load changes without component changes, and pod restart/autoscaling episodes. These controls should use the same observation duration and instrumentation as attack runs, with labels fixed before analysis. Until these runs are collected, false-positive rate, precision, specificity, and F1 are undefined; an event-positive fraction of 6/6 is not a recall estimate because its event oracle does not require a correctly attributed attack.

These three scenario classes are a proposed protocol, not verified existing runner functionality. The project repository and saved experiment set do not establish that benign-update, benign-load, and benign-restart automation has been implemented or executed. A future evaluation should pre-register the workload, contract version, event labels, and alert threshold, then repeat each class under matched workload and deployment conditions. Sample size should be selected from the desired confidence interval rather than treating three runs per class as automatically sufficient.

The current data support only that drift events were recorded during six completed attack windows. They do not bound benign alert rates or establish attack detection accuracy. The three failed SolarWinds-style injections must be fixed and rerun before that attack family can be assessed.

G. Threats to Validity

The six completed runs cover two attack families and use a run-level event-presence label; event presence does not establish that a malicious event was attributed to the injected component. The raw records have no benign controls, all-zero overhead snapshots, unresolved component identity, and no reproducibility manifest with kernel/runtime versions and image digests. Three attack injections failed. The baseline log documents a dry run: Falco and Trivy setup were skipped and Falco alert logs were missing. Trivy is a static vulnerability scanner, so a runtime detection comparison requires a separately defined task and labeling rule. PCEPS model/calibration outputs and causal ground-truth comparisons are absent. These gaps preclude claims of comparative accuracy, causal attribution quality, calibration, and operational overhead.

The causal projection also depends on caller-supplied covariates, hard-coded semantic tiers, and a DAG that includes treatment-to-outcome and covariate edges by construction. The resulting estimate is conditional on those assumptions; neither temporal precedence nor a graph path proves causation. Existing tests exercise specified abstention paths, but do not test causal correctness against intervention ground truth. The attack sample is small, run-level labels measure event presence rather than correctly attributed compromise, and failed injections are excluded. The observed event latency cannot be generalized to deployment performance without matched controls, repeated workloads, and recorded environment details.

VI. Conclusion

PHANTOM demonstrates an implemented pathway that connects SBOM-facing contract generation, eBPF event collection, typed graph maintenance, and a causal-estimation interface with explicit abstention. A local run of the core and five service test suites completed with 454 tests passed and one skipped. This verifies exercised software behaviors, not the validity of component-level causal claims in a live deployment. The experiment records show event-positive outcomes in six completed runs, but do not validate package mapping, attribution accuracy, prospective risk prediction, benign false-positive behavior, or overhead.

The immediate evaluation priorities are benign controls, per-run component-ground-truth records, repeatable SolarWinds-style injections, matched baseline output, and a trained PCEPS model evaluated on held-out labels. Kernel, Kubernetes, container runtime, tool versions, image digests, and experiment commit must be recorded with the rerun. No claim is made that the infrastructure for all these experiments is already implemented or ready to execute.

Supply-chain incidents expose a gap between declared software inventory and observed runtime activity. PHANTOM explores one way to represent that gap while making missing identity and failed causal identification visible to an analyst. Its present contribution is an implemented research prototype and an auditable account of what the available tests and six successful attack runs do—and do not—show. Controlled evaluation must establish whether its component-scoped contracts and causal pathway produce reliable attribution beyond event detection.

References

[1] CISA, "Reported Supply Chain Compromise Affecting XZ Utils Data Compression Library, CVE-2024-3094," Alert AA24-089A, Mar. 29, 2024.

[2] A. Freund, "Backdoor in upstream xz/liblzma leading to ssh server compromise," oss-security mailing list, Mar. 29, 2024. [Online]. Available: https://www.openwall.com/lists/oss-security/2024/03/29/4

[3] CISA, "Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations," Alert AA20-352A, Dec. 2020. [Online]. Available: https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a

[4] FireEye, "Highly Evasive Attacker Leverages SolarWinds Supply Chain to Compromise Multiple Global Victims With SUNBURST Backdoor," Dec. 2020.

[5] Apache Software Foundation, "Apache Log4j Security Vulnerabilities: CVE-2021-44228," Dec. 2021. [Online]. Available: https://logging.apache.org/log4j/2.x/security.html

[6] National Vulnerability Database, "CVE-2021-44228 Detail," NIST, 2021. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2021-44228

[7] Sysdig, "Falco: Cloud Native Runtime Security," CNCF project documentation. [Online]. Available: https://falco.org/docs/ (Accessed: Oct. 16, 2026).

[8] Isovalent, "Tetragon: eBPF-Based Security Observability and Runtime Enforcement," 2023. [Online]. Available: https://tetragon.io/ (Accessed: Oct. 16, 2026).

[9] Aqua Security, "Tracee: Linux Runtime Security and Forensics Using eBPF," 2022. [Online]. Available: https://github.com/aquasecurity/tracee (Accessed: Oct. 16, 2026).

[10] OWASP Foundation, "CycloneDX Specification, Version 1.4," 2022. [Online]. Available: https://cyclonedx.org/specification/archive/ (Accessed: Oct. 16, 2026).

[11] U.S. Department of Commerce, "The Minimum Elements for a Software Bill of Materials (SBOM)," NTIA, 2021. [Online]. Available: https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom

[12] Anchore Inc., "Anchore Enterprise: SBOM-Powered Software Supply Chain Security," 2023. [Online]. Available: https://anchore.com/enterprise/ (Accessed: Oct. 16, 2026).

[13] Ox Security, "Software Supply Chain Security Platform," product documentation. [Online]. Available: https://ox.security/ (Accessed: Oct. 16, 2026).

[14] Y. Chen et al., "Automatic Root Cause Analysis via Large Language Models for Cloud Incidents," in Proc. 19th European Conf. Computer Systems (EuroSys), Athens, Greece, Apr. 2024, pp. 674–688.

[15] J. Pearl, Causality: Models, Reasoning, and Inference, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2009.

[16] A. Sharma and E. Kiciman, "DoWhy: An End-to-End Library for Causal Inference," arXiv:2011.04216, 2020. [Online]. Available: https://arxiv.org/abs/2011.04216

[17] Google LLC, "Online Boutique: Cloud-Native Microservices Demo," GitHub repository. [Online]. Available: https://github.com/GoogleCloudPlatform/microservices-demo (Accessed: Oct. 16, 2026).

[18] DARPA, "Transparent Computing Program: Data and Resources," program materials. [Online]. Available: https://www.darpa.mil/research/programs/transparent-computing (Accessed: Oct. 16, 2026).

[19] AccuKnox, "KubeArmor: Container-Aware AppArmor, SELinux, and BPF Security," 2022. [Online]. Available: https://kubearmor.io/ (Accessed: Oct. 16, 2026).

[20] Anchore, "Syft: Generate a Software Bill of Materials from Container Images and Filesystems," project documentation. [Online]. Available: https://github.com/anchore/syft (Accessed: Oct. 16, 2026).

[21] Sigstore, "Cosign: Container Signing, Verification, and Storage," project documentation. [Online]. Available: https://docs.sigstore.dev/cosign/ (Accessed: Oct. 16, 2026).

[22] eBPF Foundation, "What is eBPF?" project documentation. [Online]. Available: https://ebpf.io/what-is-ebpf/ (Accessed: Oct. 16, 2026).

[23] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in Proc. SciPy, 2008, pp. 11–15.

[24] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD, 2016, pp. 785–794.

[25] J. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in Advances in Large Margin Classifiers, 1999, pp. 61–74.

[26] Package-URL Working Group, "Package URL (PURL) Specification," project documentation. [Online]. Available: https://github.com/package-url/purl-spec (Accessed: Oct. 16, 2026).

[27] Sigstore, "Cosign Documentation: Signing Containers," project documentation. [Online]. Available: https://docs.sigstore.dev/cosign/signing/signing_containers/ (Accessed: Oct. 16, 2026).

[28] Linux Foundation, "libbpf: BPF CO-RE Reference Guide," project documentation. [Online]. Available: https://github.com/libbpf/libbpf (Accessed: Oct. 16, 2026).

[29] N. Srinivasan, N. Naveen, and N. Naveen, "Bomfather: An eBPF-based Kernel-level Monitoring Framework for Accurate Identification of Unknown, Unused, and Dynamically Loaded Dependencies in Modern Software Supply Chains," arXiv:2503.02097, 2025. [Online]. Available: https://arxiv.org/abs/2503.02097.

[30] A. Crawford, E. Yakubovich, and R. Szumski, "Enforcing SBOMs through the Linux kernel with eBPF and IMA," in Proc. ACM Workshop on Software Supply Chain Security (SCORED), 2023, pp. 77–78, doi: 10.1145/3605770.3625206.